

Turbine Health Monitoring and Failure Forecasting

Abstract:

This project, titled “**Turbine Health Monitoring and Failure Forecasting**” This project develops a machine learning model to monitor wind turbine health and predict potential failures. By analyzing operational parameters such as wind speed, temperature, and vibration, the system identifies patterns leading to faults. The model enables early detection of failures, helping to reduce downtime, improve maintenance planning, and enhance overall efficiency of wind turbine operations.

INTRODUCTION

This project focuses on analyzing wind turbine performance data to identify failures and fault types. Machine learning techniques are used to predict failures and classify fault types based on input data.

Objective:

The goal of this project is to develop a predictive model for monitoring the health of wind turbines and forecasting potential failures. By analyzing key operational parameters such as WindSpeed, RotorSpeed, BladeAngle, GearboxTemperature, GeneratorTemperature, and Vibration, we aim to:

- Identify patterns or conditions that lead to failures.

- Predict specific types of failures (e.g., gearbox, generator, blade) before they occur.
- Improve turbine uptime and optimize maintenance schedules, reducing downtime and operational costs.

Tools and Technologies:

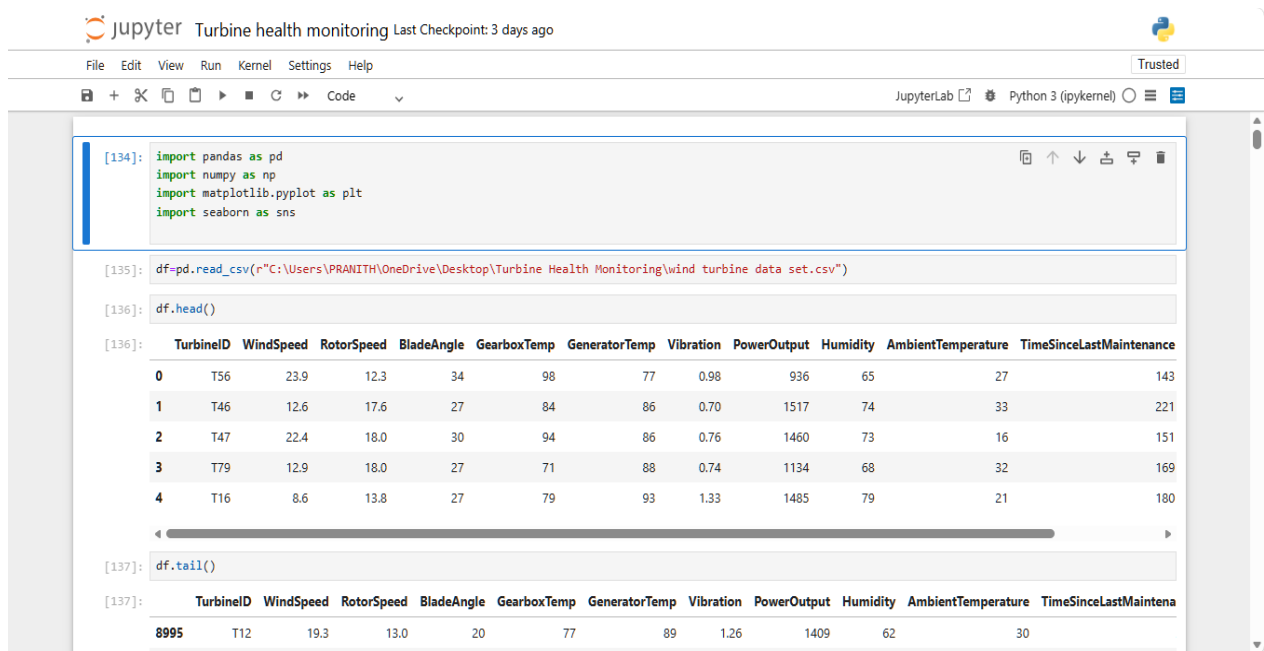
Used Python (Pandas, Jupyter Notebook) was used for data cleaning and pre-processing, (matplotlib) was used for visualization

Methodology:

- Data Exploration & Preprocessing
- Exploratory Data Analysis (EDA)
- Model Selection & Training
- Failure Type Prediction
- Hyperparameter Tuning
- Deployment
- Reporting & Visualization

Data Exploration & Preprocessing

- Load dataset (9,000 rows, multiple sensors + failure labels)
- Clean data (missing values, duplicates, consistency)
- Encode categorical variables (e.g., FaultType)



The screenshot shows a JupyterLab environment with the title 'Turbine health monitoring' and a 'Last Checkpoint: 3 days ago' status. The interface includes a menu bar (File, Edit, View, Run, Kernel, Settings, Help) and a toolbar with icons for file operations and code execution. The main area displays a code cell with the following Python code:

```
[134]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

Below the code cell, the output of the code is shown. It includes the command to read a CSV file and the first five rows of the dataset:

```
[135]: df=pd.read_csv(r"C:\Users\PRANITH\OneDrive\Desktop\Turbine Health Monitoring\wind turbine data set.csv")
[136]: df.head()
```

	TurbineID	WindSpeed	RotorSpeed	BladeAngle	GearboxTemp	GeneratorTemp	Vibration	PowerOutput	Humidity	AmbientTemperature	TimeSinceLastMaintenance
0	T56	23.9	12.3	34	98	77	0.98	936	65	27	143
1	T46	12.6	17.6	27	84	86	0.70	1517	74	33	221
2	T47	22.4	18.0	30	94	86	0.76	1460	73	16	151
3	T79	12.9	18.0	27	71	88	0.74	1134	68	32	169
4	T16	8.6	13.8	27	79	93	1.33	1485	79	21	180

Below the first five rows, there is a horizontal scrollbar. The output also shows the last row of the dataset:

```
[137]: df.tail()
```

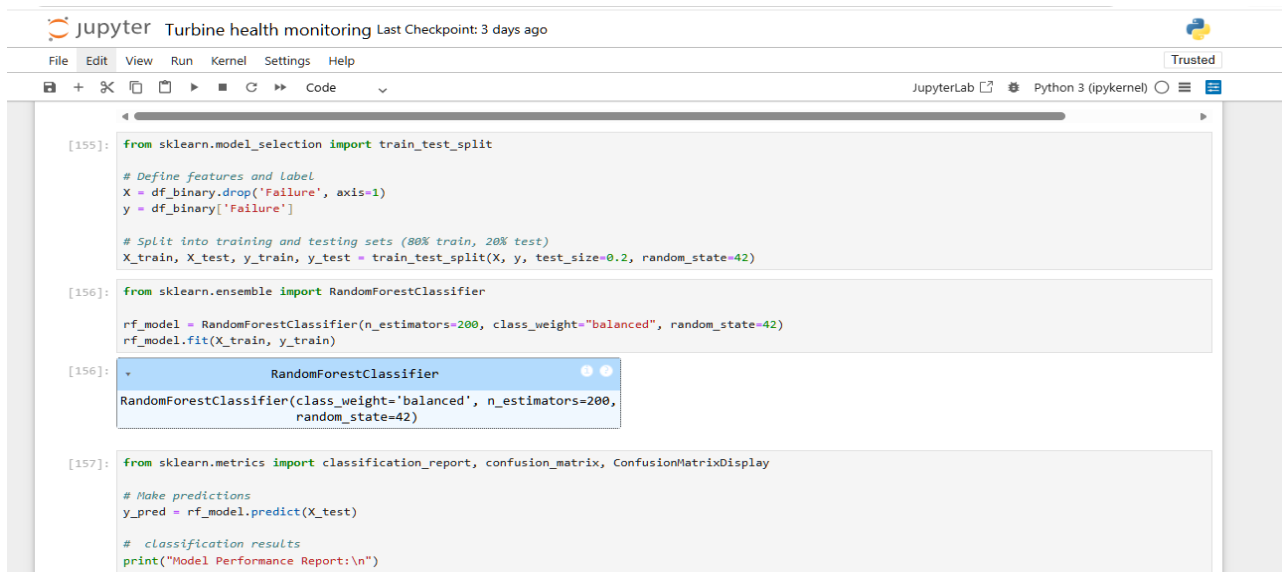
	TurbineID	WindSpeed	RotorSpeed	BladeAngle	GearboxTemp	GeneratorTemp	Vibration	PowerOutput	Humidity	AmbientTemperature	TimeSinceLastMaintenance
8995	T12	19.3	13.0	20	77	89	1.26	1409	62	30	

Exploratory Data Analysis (EDA)

- Descriptive statistics
- Correlation analysis (e.g., vibration -- failure)
- Visualizations: histograms, scatter plots, box plots
- Observations: Failures linked to temperature, vibration, maintenance gap

Model Selection & Training

- Train/Test split (80/20)
- Models tested: Decision Tree, Random Forest, XGBoost
- Metrics: Accuracy, Precision, Recall, F1, AUC-ROC
- Cross-validation for generalization



```
[155]: from sklearn.model_selection import train_test_split

# Define features and label
X = df_binary.drop('Failure', axis=1)
y = df_binary['Failure']

# Split into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

[156]: from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(n_estimators=200, class_weight="balanced", random_state=42)
rf_model.fit(X_train, y_train)

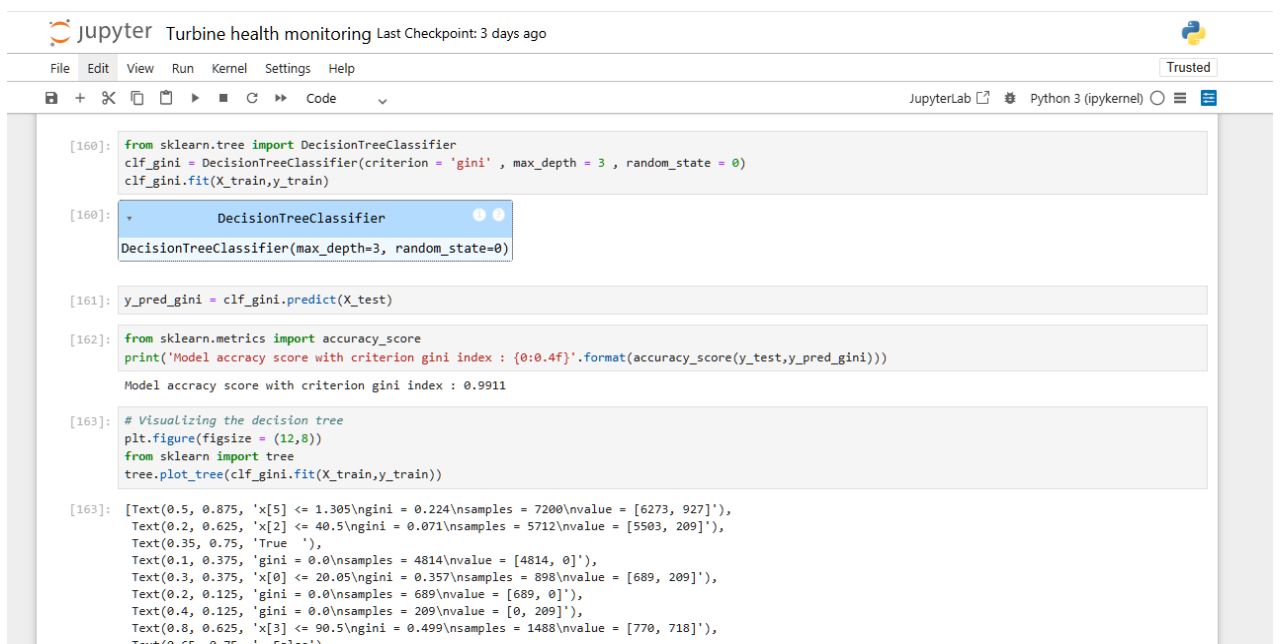
[156]: Random Forest Classifier
RandomForestClassifier(class_weight='balanced', n_estimators=200,
random_state=42)

[157]: from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay

# Make predictions
y_pred = rf_model.predict(X_test)

# classification results
print("Model Performance Report:\n")
```

Decision Tree



```
[160]: from sklearn.tree import DecisionTreeClassifier
clf_gini = DecisionTreeClassifier(criterion = 'gini' , max_depth = 3 , random_state = 0)
clf_gini.fit(X_train,y_train)

[160]: DecisionTreeClassifier
DecisionTreeClassifier(max_depth=3, random_state=0)

[161]: y_pred_gini = clf_gini.predict(X_test)

[162]: from sklearn.metrics import accuracy_score
print('Model accuracy score with criterion gini index : {0:0.4f}'.format(accuracy_score(y_test,y_pred_gini)))

Model accuracy score with criterion gini index : 0.9911

[163]: # Visualizing the decision tree
plt.figure(figsize = (12,8))
from sklearn import tree
tree.plot_tree(clf_gini.fit(X_train,y_train))

[163]: [Text(0.5, 0.875, 'x[5] <= 1.305\ngini = 0.224\nsamples = 7200\nvalue = [6273, 927]'),
Text(0.2, 0.625, 'x[2] <= 40.5\ngini = 0.071\nsamples = 5712\nvalue = [5503, 209]'),
Text(0.35, 0.75, 'True '),
Text(0.1, 0.375, 'gini = 0.0\nsamples = 4814\nvalue = [4814, 0]'),
Text(0.3, 0.375, 'x[0] <= 20.05\ngini = 0.357\nsamples = 898\nvalue = [689, 209]'),
Text(0.2, 0.125, 'gini = 0.0\nsamples = 689\nvalue = [689, 0]'),
Text(0.4, 0.125, 'gini = 0.0\nsamples = 209\nvalue = [0, 209]'),
Text(0.8, 0.625, 'x[3] <= 90.5\ngini = 0.499\nsamples = 1488\nvalue = [770, 718]'),
Text(0.65, 0.75, 'False')]
```

XG Boost

```
jupyter Turbine health monitoring Last Checkpoint: 3 days ago

File Edit View Run Kernel Settings Help Trusted

JupyterLab Python 3 (ipykernel)

[175]: # Importing Libraries
from xgboost import XGBClassifier
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay
import matplotlib.pyplot as plt

#Creating and training the XGBoost model
model = XGBClassifier(eval_metric='logloss', random_state=42)
model.fit(X_train, y_train)

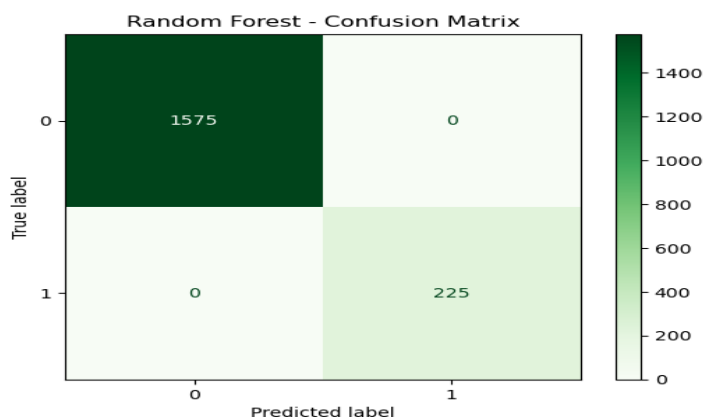
[175]: XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric='logloss',
              feature_types=None, feature_weights=None, gamma=None,
              grow_policy=None, importance_type=None,
              interaction_constraints=None, learning_rate=None, max_bin=None,
              max_cat_threshold=None, max_cat_to_onehot=None,
              max_delta_step=None, max_depth=None, max_leaves=None,
              min_child_weight=None, missing=nan, monotone_constraints=None,
              multi_strategy=None, n_estimators=None, n_jobs=None,

[178]: # predictions on the test data
y_pred_XG = model.predict(X_test)

# classification performance
print("XGBoost Classification Report:")
```

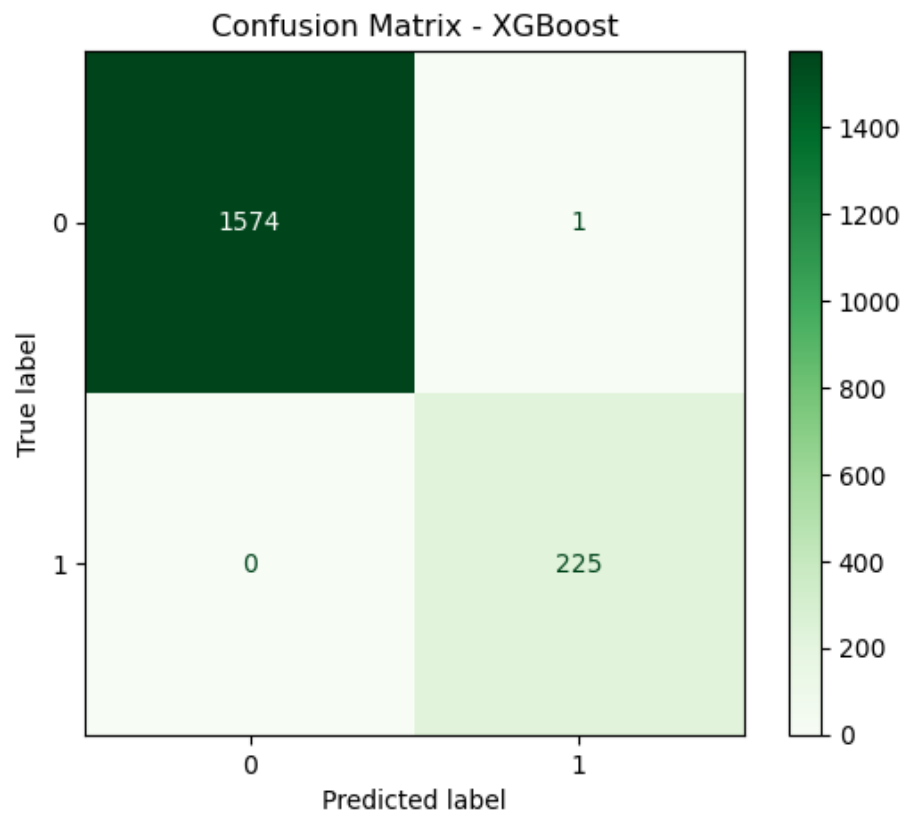
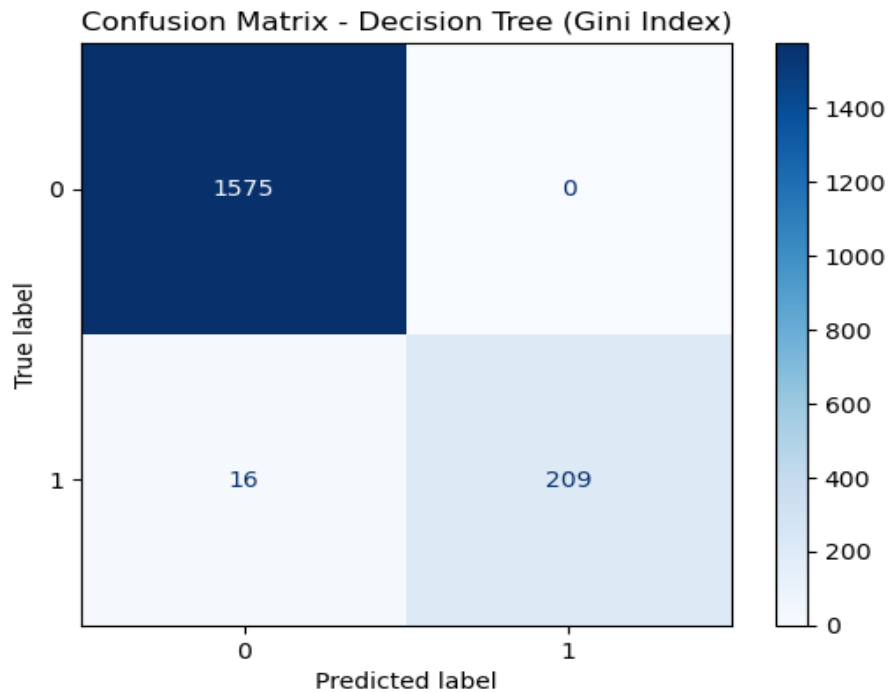
Failure Type Prediction

- Multiclass classification: FaultType (Gearbox, Generator, Blades)
- Models: Random Forest, XGBoost
- Evaluated using confusion matrix
- Insight : Failures due to multi-factor stress



Confusion matrix of decision trees

/xgboost



Deployment

- Model saved as Pickle (.pkl)
- Deployed with Streamlit (app.py)
- User inputs sensor values → Predicts Healthy / Need Maintenance
- Alerts triggered when risk is high

```
181]: import pickle
pickle.dump(rf_model, open("RF_turbine_model.pkl", "wb"))
print(" Model trained and saved as RF_turbine_model.pkl")

Model trained and saved as RF_turbine_model.pkl
```

```
[ ]:
```

----- Turbine Health Monitoring ----- ----- and Failure Forecasting -----

Predict turbine failure risk using sensor data

Wind Speed (m/s)

8.60

- +

Rotor Speed (RPM)

13.80

- +

Blade Angle (degrees)

27.00

- +

Gearbox Temperature (°C)

79.00

- +

Generator Temperature (°C)

Power Output (kW)

1485.00

- +

Humidity (%)

79.00

- +

Ambient Temperature (°C)

21.00

- +

Time Since Last Maintenance (days)

180.00

- +

Predict

Raw model prediction: 0

✓ No Failure Detected - Turbine is Healthy

Conclusion

- Predictive maintenance for turbines is feasible and effective
- Random Forest chosen for accuracy, stability, and interpretability
- Next steps:
 - - Integrate live streaming data
 - - Enhance IoT dashboards
 - - Refine thresholds for real-time alarms
- The project successfully analyzed turbine failures and fault types. The machine learning model provided accurate predictions, and visualization tools helped in understanding the data clearly. The system can help in predictive maintenance of wind turbines.